

Networks & System Security

SIL765 & SIL7165 - Quiz 2 - 03 February 2026

Name: _____

Entry Number: _____

Select Course Code: ☐ SIL765 ☐ SIL7165

Instructions:

- Each question is of **multiple-select** type.
- One, multiple, or all options may be correct.
- There will be no partial credit unless explicitly stated by the instructor.
- To indicate your choice, please place a check mark (✓) next to the selected option. No other symbols will be accepted.

Q1

A browser vulnerability allows a heap-based buffer overflow in a JavaScript-accessible object. An attacker uses JavaScript to perform heap spraying before triggering the overflow.

The system enforces the following defenses:

- Heap Address Space Layout Randomization (ASLR) is enabled
- A strict W^X (Write XOR Execute) policy is enforced
- No memory disclosure vulnerabilities are present

Which of the following conclusions logically follow? Select **all** that apply.

- (A) Heap spraying can still increase exploit reliability despite ASLR
- (B) ASLR eliminates the usefulness of heap spraying entirely
- (C) Heap spraying relies on probabilistic coverage rather than exact address knowledge
- (D) W^X alone is sufficient to prevent control-flow hijacking
- (E) Control-flow redirection may still succeed even if injected code cannot execute

- **Correct options: A, C, E**
- **A) Heap spraying increases probability of landing in attacker-controlled memory — TRUE.** Heap spraying fills large regions of memory with attacker-controlled data, increasing the likelihood that a corrupted control-flow target lands inside a useful payload despite address randomization.
- **C) It relies on probabilistic memory coverage rather than exact addresses — TRUE.** Heap spraying does not require precise address knowledge; instead, it relies on covering a large fraction of the address space to win probabilistically.
- **E) Control flow can still be redirected even if injected code cannot execute — TRUE.** Even when data execution is prevented (e.g., NX), heap spraying can support control-flow redirection to reusable code or existing executable regions.

Q2

Consider a system where:

- W^X is strictly enforced

- ASLR is disabled
- The attacker can overwrite a saved return address on the stack

Which attack strategies remain logically feasible under these constraints? Select **all** that apply.

- (A) Injecting shellcode directly onto the stack
- (B) Redirecting control flow to existing library functions
- (C) Reusing executable code sequences already present in memory
- (D) Executing shellcode placed on the heap via heap spraying
- (E) Constructing arbitrary behavior using chained return instructions

- **Correct options: B, C, E**
- **B) Return-to-libc redirects execution to existing executable library code — TRUE.** Return-to-libc attacks reuse trusted library functions instead of injecting new code, bypassing non-executable memory protections.
- **C) Code-reuse attacks use instructions already present in executable memory — TRUE.** Gadgets are extracted from existing executable regions, allowing attackers to construct arbitrary behavior without code injection.
- **E) Chaining return instructions enables arbitrary behavior without injecting code — TRUE.** By chaining return addresses (ROP), attackers can simulate complex logic using only existing instructions.

Q3

An operating system deploys Address Space Layout Randomization (ASLR) to protect against memory corruption exploits. The attacker has no direct control over ASLR configuration but can repeatedly interact with the target process.

Which properties of ASLR can still be exploited in practice? Select **all** that apply.

- (A) Objects within the same memory region maintain fixed relative offsets
- (B) Forked processes may reuse identical randomized layouts
- (C) Leakage of a single address may reveal all addresses within that region
- (D) ASLR randomization is applied only at compile time
- (E) Alignment constraints reduce entropy in lower-order address bits

- **Correct options: A, B, C, E**
- **A) Relative offsets inside a memory region remain fixed — TRUE.** While ASLR randomizes base addresses, relative offsets within a region (e.g., inside a shared library) remain constant.
- **B) Forked processes may inherit the same randomized layout — TRUE.** Child processes created via `fork()` often inherit the parent's memory layout, reducing ASLR effectiveness.
- **C) One leaked address can reveal the base of that region — TRUE.** A single information leak can expose the base address of a module, enabling calculation of all internal offsets.
- **E) Alignment reduces entropy in lower address bits — TRUE.** Page and alignment constraints reduce randomness, especially in lower bits, weakening ASLR entropy.

Q4

A privileged program checks access permissions on a file and later opens the same file for writing. The check and the use are performed in separate system calls.

Which conditions make this vulnerability practically exploitable? Select **all** that apply.

- (A) The attacker can influence scheduling between the check and the use
- (B) The file name can be replaced using symbolic links
- (C) The vulnerable program executes with elevated privileges
- (D) The check and the use are combined into a single atomic system call
- (E) The attacker can repeatedly trigger the vulnerable code path

- **Correct options: A, B, C, E**
- **A) Exploitation requires winning the race between check and use — TRUE.** TOCTOU vulnerabilities arise when an attacker changes a resource between the time it is checked and the time it is used.
- **B) Symlinks allow redirection to sensitive files — TRUE.** Symbolic links can redirect privileged file accesses to attacker-chosen targets after the initial check.
- **C) Elevated privileges make the attack meaningful — TRUE.** These attacks are most dangerous when the vulnerable program runs with higher privileges than the attacker.
- **E) Repeated attempts increase success probability — TRUE.** Since race conditions are timing-dependent, repeated attempts significantly increase the likelihood of exploitation.

Q5

A program validates input size using integer arithmetic before copying user input into a fixed-size buffer. The validation logic appears correct during manual inspection.

Which scenarios logically allow an integer overflow to result in a buffer overflow? Select **all** that apply.

- (A) Signed integers are used for bounds checks while unsigned values are used for memory operations
- (B) Length validation is performed only after memory allocation
- (C) Arithmetic overflow occurs before the bounds comparison
- (D) The buffer size is fully determined at compile time
- (E) Implicit integer promotion changes the width of operands during computation

- **Correct options: A, C, E**
- **A) Signed/unsigned mismatch can bypass bounds checks — TRUE.** Implicit type conversions can cause negative values to be interpreted as large unsigned integers, bypassing checks.
- **C) Overflow before comparison invalidates the check — TRUE.** Arithmetic overflow can occur before validation, causing comparisons to behave incorrectly.
- **E) Integer promotion can silently change values — TRUE.** Automatic integer promotion in expressions may alter values in unexpected ways, leading to subtle vulnerabilities.

Q6

A runtime defense monitors heap memory to detect heap spraying attacks. The detection mechanism relies on statistical properties of memory contents.

Which factors increase the likelihood that such an attack will be detected? Select **all** that apply.

- (A) Large regions of memory containing highly repetitive instruction patterns
- (B) Numerous contiguous allocations containing executable payloads
- (C) Use of shorter NOP sleds instead of longer ones
- (D) Sprayed objects closely mimicking legitimate application data
- (E) Shellcode allocated statically at compile time

- **Correct options: A, B**
- **A) Repetitive instruction patterns indicate sprayed shellcode — TRUE.** Heap spraying often uses repeated instructions (e.g., NOP sleds) to increase landing reliability.
- **B) Large contiguous payload regions are suspicious — TRUE.** Abnormally large contiguous memory regions filled with uniform data are a common indicator of heap spraying.

Q7

A program processes user input and logs it using the following code fragment:

```
void log_message(char *user_input) {
    printf(user_input);
}
```

An attacker can fully control the contents of `user_input`. The program does not crash during normal execution.

Which of the following outcomes are logically possible? Select **all** that apply.

- (A) Disclosure of stack or memory contents
- (B) Modification of memory values at attacker-chosen locations
- (C) Guaranteed execution of attacker-supplied code
- (D) Denial-of-service due to invalid memory access
- (E) Silent corruption of program state without immediate crashes

- **Correct options: A, B, D, E**
- **A) Format specifiers can leak stack or memory contents — TRUE.** Unsafe format strings allow attackers to read arbitrary memory values.
- **B) %n allows writing to memory — TRUE.** The %n specifier writes the number of printed bytes to an attacker- controlled address.
- **D) Invalid memory access can crash the program — TRUE.** Malformed format strings can dereference invalid addresses, causing crashes.
- **E) %n can corrupt program state without immediate crashes — TRUE.** Silent memory corruption may occur, leading to delayed or subtle failures.

Q8

A system uses Address Space Layout Randomization (ASLR) to mitigate memory corruption exploits. Two versions of the same program are compiled and executed on this system:

- Program A is compiled **without** Position Independent Executable (PIE)
- Program B is compiled **with** PIE enabled

Both programs are dynamically linked and executed multiple times.

Which of the following statements correctly describe the differences in behavior between Program A and Program B? Select **all** that apply.

- (A) In Program A, the main executable is loaded at a fixed virtual address across executions
- (B) In Program B, the base address of the main executable can change between executions
- (C) ASLR randomizes the location of shared libraries regardless of whether PIE is enabled
- (D) PIE affects whether stack and heap addresses are randomized
- (E) Without PIE, ASLR provides no protection against code-reuse attacks in the main executable

- **Correct options: A, B, C**
- **A) Non-PIE executables load at fixed addresses — TRUE.** Without PIE, the main executable is loaded at a predictable address, weakening ASLR.
- **B) PIE enables base address randomization — TRUE.** Position-Independent Executables allow the entire binary to be relocated, increasing ASLR effectiveness.
- **C) Shared libraries are randomized regardless of PIE — TRUE.** Shared libraries are position-independent by design and benefit from ASLR even when the main binary is not PIE.

Q9. Cross-Origin Requests, Cookies, and CSRF

A user is logged into `bank.com`. The site uses cookie-based authentication and performs money transfers via:

POST `https://bank.com/transfer`

The site has deployed all of the following defenses:

- Cookies have attributes: `Secure`; `HttpOnly`; `SameSite=Lax`
- All pages are served over HTTPS with HSTS enabled
- No CSRF tokens are used
- No custom headers are required
- CORS is not enabled

An attacker hosts a malicious page on `evil.com` containing:

```
<form action="https://bank.com/transfer" method="POST">
  <input type="hidden" name="to" value="attacker">
  <input type="hidden" name="amount" value="1000">
  <input type="submit">
</form>
<script>document.forms[0].submit()</script>
```

Which statement is correct?

- A) The attack succeeds because cookies are always sent to the target domain.
- B) The attack fails because `SameSite=Lax` blocks cookies on cross-site POST requests.
- C) The attack fails because HSTS prevents cross-origin requests.
- D) The attack succeeds because `SameSite=Lax` allows cookies on cross-site POST requests.

- **Correct option: B**
- **Clarifying the SameSite=Lax policy.** The SameSite=Lax cookie policy enforces the following rules:
 - All *same-site* requests are allowed to carry cookies, regardless of method.
 - For *cross-site* requests, cookies are sent *only* for **top-level navigational requests**, such as GET requests typically initiated by direct user interaction (e.g., clicking a link).
 - Non-navigational cross-site requests—such as automatic form submissions, background requests, or subresource loads—do *not* include cookies.
- **A) The attack succeeds because cookies are always sent to the target domain — FALSE.** While cookies were historically sent with all requests to the target domain, modern browsers enforce the SameSite attribute. Under SameSite=Lax, cookies are *not* always sent; their inclusion depends on whether the request is same-site or a permitted top-level cross-site navigation.
- **B) The attack fails because SameSite=Lax blocks cookies on cross-site POST requests — TRUE.** The attack uses an automatically submitted cross-site POST form. This request is:
 - Cross-site, and
 - Not a top-level navigational request initiated by user interaction.

Therefore, under SameSite=Lax, the browser does *not* attach the session cookie. Without the authentication cookie, the request fails and the transfer is rejected.

- **C) The attack fails because HSTS prevents cross-origin requests — FALSE.** HTTP Strict Transport Security (HSTS) only enforces the use of HTTPS instead of HTTP. It does not restrict cross-origin requests and provides no protection against CSRF attacks.
- **D) The attack succeeds because SameSite=Lax allows cookies on cross-site POST requests — FALSE.** This statement misinterprets the exception provided by SameSite=Lax. While SameSite=Lax does allow cookies on certain cross-site requests, this exception applies *only* to **top-level navigational requests**, typically safe methods such as GET triggered by user action. Automatically submitted cross-site POST requests do not qualify as navigational requests, so cookies are correctly withheld.

Q10. Cross-Origin Embedding vs Inspection

A page hosted at `attacker.com` embeds the following resource:

```

```

Assume the victim is logged into `bank.com`.
Which statements are correct?

- A) The browser will send cookies for `bank.com` along with the request.
- B) The JavaScript running on `attacker.com` can read the HTTP response body.
- C) The request may still cause a state change on `bank.com`.
- D) This violates the DOM Same-Origin Policy and the browser blocks the request.
- E) This behavior can be exploited even without reading the response.

- **Correct options: A, C, E**
- **A) The browser will send cookies for bank.com along with the request — TRUE.** Cookies are sent based on the *target origin*, not the page that initiated the request. If the request is made to **bank.com** and the cookie is in-scope for that domain, the browser automatically attaches it, even if the request originated from **attacker.com**.
- **B) JavaScript on attacker.com can read the response body — FALSE.** The DOM Same-Origin Policy allows cross-origin resource embedding, but it prevents scripts from inspecting the contents of responses from other origins. An `<img>` request does not expose response bytes to attacker-controlled JavaScript.
- **C) The request may still cause a state change on bank.com — TRUE.** Same-Origin Policy restricts reading responses, not sending requests. If the requested endpoint performs side effects (e.g., account updates or transfers), a cross-origin request can still trigger those effects. This is the fundamental observation that enables CSRF-style attacks.
- **D) This violates SOP and the browser blocks the request — FALSE.** Browsers explicitly allow cross-origin embedding of resources such as images, scripts, and iframes. SOP blocks *inspection*, not request issuance.
- **E) This behavior can be exploited even without reading the response — TRUE.** Many web attacks only require the attacker to cause an authenticated request to be sent; observing the response is unnecessary. This weakness is explicitly highlighted in the web security model as a core limitation of SOP.

Q11. Sub-Resource Integrity vs CSP vs HTTPS

A website loads a third-party library:

```
<script src="https://cdn.example/lib.js"></script>
```

The site deploys:

- HTTPS with a valid certificate
- HSTS enabled
- Content-Security-Policy: `script-src https://cdn.example`
- No SRI attribute

Later, the CDN is compromised and serves malicious JavaScript from the same URL. Which statement is correct?

- A) HTTPS prevents execution because the CDN was compromised.
- B) CSP blocks execution because the script origin is no longer trusted.
- C) The malicious script executes successfully.
- D) HSTS blocks the compromised script.

- **Correct option: C**
- **A) HTTPS prevents execution because the CDN was compromised — FALSE.** HTTPS protects against network-level attackers such as eavesdroppers and man-in-the-middle adversaries. It does not protect against a compromised server that legitimately serves malicious content over HTTPS.
- **B) CSP blocks execution because the script origin is no longer trusted — FALSE.** CSP enforces origin-based trust, not content integrity. Because the policy explicitly whitelists `https://cdn.example`, scripts served from that origin are permitted to execute, regardless of whether their contents have changed.
- **C) The malicious script executes successfully — TRUE.** The Content Security Policy explicitly allows scripts from `https://cdn.example`, and the compromised script is still delivered from that trusted origin. Since no Sub-Resource Integrity (SRI) hash is specified, the browser has no mechanism to detect that the script contents have been altered. As a result, the malicious JavaScript executes successfully.
- **D) HSTS blocks the compromised script — FALSE.** HTTP Strict Transport Security (HSTS) only enforces that connections use HTTPS instead of HTTP. It does not inspect or validate the correctness or integrity of JavaScript resources.

Q12. CORS and Pre-Flight Semantics

Which situations will trigger a CORS pre-flight request?

- A) A GET request made via ``.
- B) A POST request sending JSON via `XMLHttpRequest`.
- C) A request with a custom header such as `X-Requested-With`.
- D) A standard HTML form POST with content-type `application/x-www-form-urlencoded`.
- E) A request using only simple headers and methods.

- **Correct options: B, C**
- **Preflight rule.** A CORS pre-flight request is triggered when a request would have been impossible to send without JavaScript, i.e., when it is a *non-simple* request.
- **A) GET via `` — FALSE.** This is a classic *simple* cross-origin embedding request. The browser sends it directly without issuing a pre-flight request.
- **B) POST sending JSON via `XMLHttpRequest` — TRUE.** A JSON POST (e.g., `Content-Type: application/json`) cannot be generated by a standard HTML form. Because it is a non-simple request, the browser must first issue an `OPTIONS` pre-flight request.
- **C) Request with a custom header such as `X-Requested-With` — TRUE.** Custom headers are not permitted in simple requests. Their presence forces the browser to perform a pre-flight check to determine whether the server authorizes the request.
- **D) HTML form POST with `application/x-www-form-urlencoded` — FALSE.** This request falls squarely within the definition of a simple request and therefore does not trigger a pre-flight.
- **E) Request using only simple headers and methods — FALSE.** This is precisely the scenario in which a pre-flight request is *not* triggered.

Q13. Content Security Policy (CSP)

Which security guarantees are correctly provided by CSP when properly configured?

- A) Prevents inline JavaScript execution when `script-src 'self'` is used.
- B) Prevents clickjacking when `frame-ancestors 'none'` is set.
- C) Prevents SQL injection at the server.
- D) Can enforce Sub-Resource Integrity for scripts and styles.

E) Automatically encrypts all network traffic.

• **Correct options: A, B, D**

- **A) Prevents inline JavaScript execution when script-src 'self' is used — TRUE.** When `script-src 'self'` is specified, CSP disallows inline scripts (such as `<script>alert(1)</script>`) unless explicitly allowed via `'unsafe-inline'` or nonces/hashes. This is a core CSP defense against XSS attacks.
- **B) Prevents clickjacking when frame-ancestors 'none' is set — TRUE.** The `frame-ancestors` directive controls which origins are allowed to embed the page in frames or iframes. Setting it to `'none'` prevents all framing, effectively mitigating clickjacking attacks.
- **C) Prevents SQL injection at the server — FALSE.** CSP is a *browser-side* security mechanism. SQL injection is a server-side vulnerability caused by improper input handling and query construction, which CSP cannot prevent.
- **D) Can enforce Sub-Resource Integrity for scripts and styles — TRUE.** CSP can require that scripts and styles use SRI hashes via the `require-sri-for` directive. This ensures that external resources are loaded only if their integrity hashes match the expected values.
- **E) Automatically encrypts all network traffic — FALSE.** CSP does not provide transport-layer encryption. Network encryption is provided by HTTPS/TLS, while CSP focuses on controlling resource loading and execution behavior within the browser.

Q14. Cookie SOP Collision and Third-Party JavaScript

A user visits:

`https://bank.com/dashboard`

The page includes a third-party analytics script:

```
<script src="https://analytics.com/track.js"></script>
```

The authentication cookie is set as:

```
Set-Cookie: session=ABC123; Domain=bank.com; Path=/; Secure
```

No `HttpOnly` flag is set.

Assume:

- The analytics script executes inside the `bank.com` page context.
- DOM Same-Origin Policy is enforced correctly.
- No CSP restrictions are deployed.

Which statement is correct?

- A) The analytics script cannot read the cookie because it originates from `analytics.com`.
- B) The analytics script can read and exfiltrate the cookie.
- C) The cookie is inaccessible because cookies obey stricter SOP than JavaScript.
- D) The cookie is protected because it is marked `Secure`.

- **Correct option: B**
- **A) The analytics script cannot read the cookie because it originates from analytics.com — FALSE.** The origin from which a script is fetched does *not* determine its execution privileges. JavaScript executes with the permissions of the *embedding page*, not the script's source origin. Since the script is included in a **bank.com** page, it executes in the **bank.com** origin.
- **B) The analytics script can read and exfiltrate the cookie — TRUE.** This scenario illustrates a classic *Same-Origin Policy (SOP) policy collision*. Although the script is fetched from **analytics.com**, it executes within the context of the embedding page, i.e., the **bank.com** origin. The authentication cookie belongs to **bank.com** and is in scope (**Domain=bank.com; Path=/**). Because the **HttpOnly** flag is *not* set, the cookie is accessible to JavaScript via **document.cookie**. The analytics script can therefore read the cookie and exfiltrate it (e.g., using an image beacon or an XMLHttpRequest).
- **C) The cookie is inaccessible because cookies obey stricter SOP than JavaScript — FALSE.** Cookies follow a different origin model based on domain and path, not the DOM Same-Origin Policy. SOP collisions occur precisely because embedded JavaScript executes with page privileges while cookies are scoped independently.
- **D) The cookie is protected because it is marked Secure — FALSE.** The **Secure** flag only prevents cookies from being sent over unencrypted HTTP connections. It does not restrict JavaScript access and therefore does not prevent cookie theft in this scenario.

Q15. Sub-Resource Integrity (SRI)

Which statements about SRI are correct?

- A) The browser hashes the fetched resource and compares it with the declared hash.
- B) If the hash mismatches, the script is still executed with a warning.
- C) SRI protects against compromised CDNs serving malicious scripts.
- D) SRI can be enforced site-wide using CSP.
- E) SRI guarantees freshness of third-party content.

- **Correct options: A, C, D**
- **A) Browser hashes fetched resource and compares to declared hash — TRUE.** This is exactly the Sub-Resource Integrity (SRI) mechanism: the browser computes the hash of the fetched resource and compares it against the hash specified in the **integrity** attribute.
- **B) If the hash mismatches, the script is still executed with a warning — FALSE.** When the computed hash does not match the declared value, the browser *refuses to execute* the script or stylesheet and reports an error.
- **C) SRI protects against compromised CDNs serving malicious scripts — TRUE.** As illustrated by real-world CDN compromise incidents (e.g., MaxCDN), SRI prevents execution of modified third-party resources, even when they are served from an otherwise trusted origin.
- **D) SRI can be enforced site-wide using CSP — TRUE.** Content Security Policy can require SRI for scripts and styles using the **require-sri-for** directive, ensuring that all included resources provide valid integrity hashes.
- **E) SRI guarantees freshness of third-party content — FALSE.** SRI guarantees *integrity*, not freshness. If a site continues to reference the same hash, the browser will continue to accept that exact version of the resource, even if newer versions exist.